

Computer Algorithms (Book)

Ellis  
Horowitz

Sartaj  
Sahni

Sanguthanar  
Rajasekaran

CS - 402

ADA

NOTES

UNIT - I

INTRODUCTION  
TO  
ALGORITHMS

By: Mr. Dayanand Padav  
Assistant Professor, CSE Dept.

①

What is an algorithm?

It is a step by step procedure for solving a computational problem.

What is a program?

It is also a step by step procedure for solving a computational problem.

## Algorithms vs Program (2)

### Algorithm

1. Design
2. Domain Expert
3. Any Language
4. HW & OS Independent
5. Analyze

### Program

1. Implementation
2. Programmer
3. Prog. Lang
4. HW & OS Dependent
5. Testing

## Characteristics of Algorithm

1. Input — 0 or more
2. Output — at least one o/p
3. Definiteness —  $\sqrt{-1} \times$
4. Finiteness —  $\infty \times$
5. Effectiveness

## How to write an Algorithm

Algorithm swap (a, b)

Begin

temp = a;

a = b;

b = temp;

end

## How to analyze an Algorithm

priori Analysis  
(Before implementation)

1. Algorithm

2. Language independent

3. H/W independent

4. Time & Space  
function

posteriori Testing  
(After implementation)

1. Program

2. Language dependent

3. H/W dependent

4. Watch time & Bytes

# How to Analyze an Algorithm

1. Time
2. Space
3. N/w Consumption & Utilization
4. Power Consumption
5. CPU Registers (CPU utilization)

Algorithm swap (a, b)

|           | <u>Time</u>              | <u>space</u>             |
|-----------|--------------------------|--------------------------|
| {         |                          |                          |
| temp = a; | 1 unit                   | a - 1                    |
| a = b;    | 1 unit                   | b - 1                    |
| b = temp; | 1 unit                   | temp - 1                 |
| }         |                          |                          |
|           | $f(n) = 3$ units         | $S(n) = 3$ words         |
|           | <u><math>O(1)</math></u> | <u><math>O(1)</math></u> |

$x = 5 * a + 6 * b$  — 1 (4 statements)



# ① Frequency Count Method ①

A = 

|   |   |   |   |   |
|---|---|---|---|---|
| 8 | 3 | 9 | 7 | 2 |
| 0 | 1 | 2 | 3 | 4 |

int n = 5

i = 0

i = 1

i = 2

i = 3

i = 4

i = 5

int main ()  
Algorithm Sum (A, n)

{ int A[] = {1, 2, 3, 4, 5};  
int n = 5  
int S = 0;

for (  $\frac{i=0}{1}; \frac{i \leq n}{n+1}; \frac{i++}{n} )$  {

S = S + A[i];

}  
printf ("sum = %d", S);  
return 0;

f(n) = 2n + 3

O(n)

space

A - n

n - 1

S - 1

i = 1

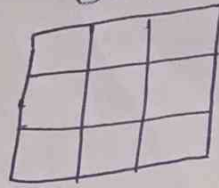
Sch) n + 3

O(n)



(5.1) ~~5.1~~

② Frequency Count Method  $n \times n$   
 $3 \times 3$



Algorithm Add(A, B, n)

```
{
  for (i=0; i ≤ n; i++) ——— n+1
  {
    for (j=0; j ≤ n; j++) ——— n(n+1)
    {
      C[i][j] = A[i][j] + B[i][j]; ——— n n × n
    }
  }
}
```

$$f(n) = 2n^2 + 2n + 1$$

$$O(n^2)$$

space

$$A - n^2$$

$$B - n^2$$

$$C - n^2$$

$$n - 1$$

$$i - 1$$

$$j - 1$$

$$s(n) = 3n^2 + 3$$

$$O(n^2)$$

### ③ Frequency Count method 5.2 ~~5.2~~

$$p = 0;$$

for ( $i = 1; p \leq n; i++$ )

{

$$p = p + i;$$

}

Assume  $p > n$

$$\therefore p = \frac{k(k+1)}{2}$$

$$\frac{k(k+1)}{2} > n$$

$$k^2 > n$$

$$\boxed{k > \sqrt{n}}$$

$$\boxed{O(\sqrt{n})}$$

| i  | p                           |
|----|-----------------------------|
| 1  | $0 + 1 = 1$                 |
| 2  | $1 + 2 = 3$                 |
| 3  | $1 + 2 + 3 = 6$             |
| 4  | $1 + 2 + 3 + 4 = 10$        |
| 5  |                             |
| 6  |                             |
| 7  |                             |
| 8  |                             |
| 9  |                             |
| 10 |                             |
| K  | $1 + 2 + 3 + 4 + \dots + k$ |

(5.3)

# ④ Frequency Count Method

```
for (i=1; i<n; i=i*2)
{
    stmt;
}
```

Assume  $i \geq n$

$$\therefore i = 2^k$$

$$\therefore 2^k \geq n$$

$$\underline{2^k = n}$$

$$\boxed{k = \log_2 n}$$

| i                    |
|----------------------|
| 1                    |
| $1 \times 2 = 2$     |
| $2 \times 2 = 2^2$   |
| $2^2 \times 2 = 2^3$ |
| $2^3 \times 2 = 2^4$ |
| ...                  |
| $2^k$                |

$$\boxed{O(\log_2 n)}$$

(5.4) Frequency count method

```
for (i=0; i<n; i++) ————— n
{
    for (j=1; j<n; j=j*2) ————— n * log n
    {
        stmt; ————— n * log n
    }
}
```

---

$$2n \log n + n$$

$$O(n \log n)$$

---

Summary

for

(5.5)

## Summary

for ( $i = 0; i < n; i++$ ) —  $O(n)$

for ( $i = 0; i < n; i = i + 2$ ) —  ~~$O(n)$~~   $O(n)$

for ( $i = n; i > 1; i--$ ) —  $O(n)$

for ( $i = 1; i < n; i = i * 2$ ) —  $O(\log_2 n)$

for ( $i = 1; i < n; i = i * 3$ ) —  $O(\log_3 n)$

for ( $i = n; i > 1; i = i / 2$ ) —  $O(\log_2 n)$



$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < n^n$$

## Asymptotic Notations

(6)

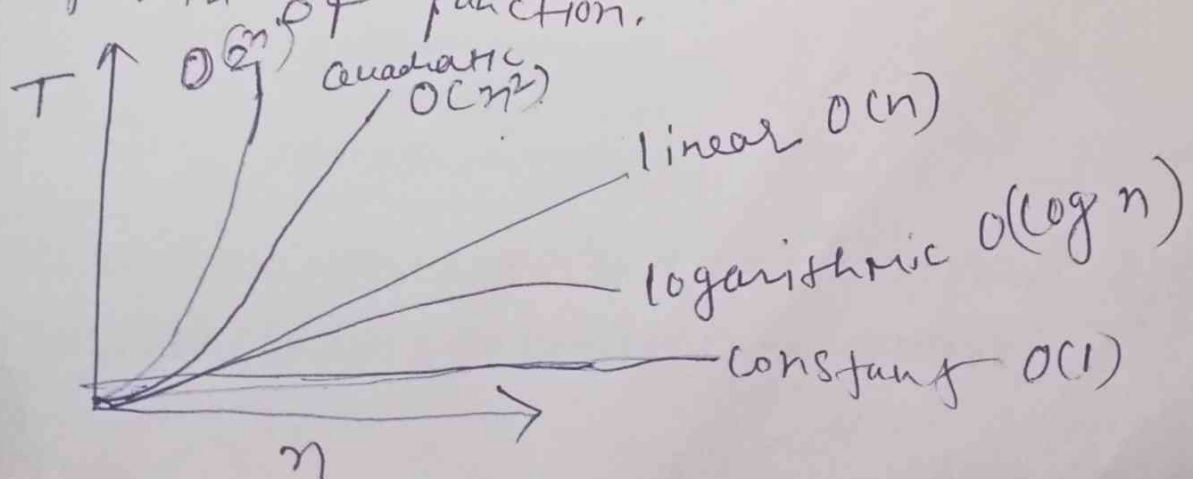
This topic comes from mathematics.

To calculate the time complexity of an Algorithm we need a function

That's why we are using the

Asymptotic notation to represent time complexity of our algorithm in the form of mathematical notations that is Asymptotic notation.

Asymptotic notations are mathematical notations for describing the behaviour of algorithm and determine the rate of growth of function.





$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < \dots < n^n$$

## Asymptotic Notations

(6.1)

$O$  big-oh (Upper bound)

$\Omega$  omega (Lower bound)

~~Useful 1~~  $\Theta$  Theta (Average bound) (~~Tight Bound~~)

\* We write  $O(1)$  to mean a computing time that is a constant.

\*  $O(\log n)$  Logarithmic

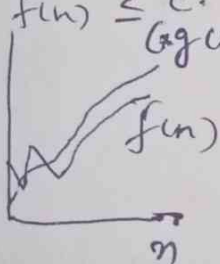
\*  $O(n)$  is called Linear

\*  $O(n^2)$  is called quadratic

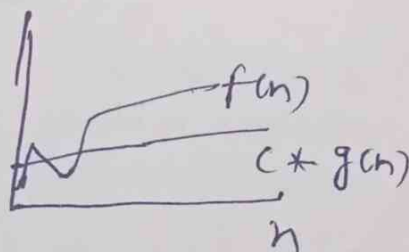
\*  $O(n^3)$  is called cubic

\*  $O(2^n)$  is called exponential

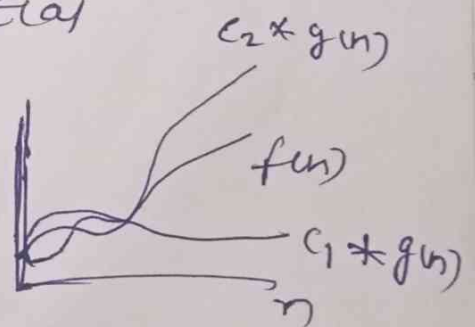
$$f(n) = O(g(n)) \\ f(n) \leq c \cdot g(n)$$



$O$   
(Upper)



$\Omega$   
(Lower)



$\Theta$   
(Average)

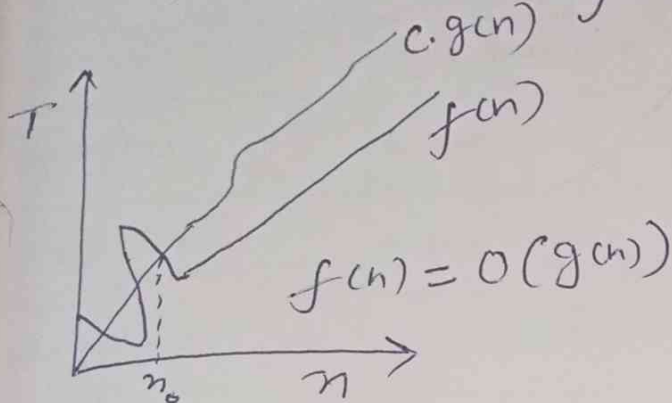
$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < \dots < n^n$$

Lower bound      Avg bound      Upper bound

Big - Oh      ①      ②      ③

6.2

The function  $f(n) = O(g(n))$



if  $\exists$  (there exist)  
+ve constants  
 $c \neq 0$  &  $n_0$

Such that  $f(n) \leq c * g(n) \forall n \geq n_0$   
↑  
for all

e.g.:  $f(n) = 2n + 3$

$$2n + 3 \leq 10n \quad n \geq 1$$

↑      ↑      ↑  
 $f(n)$        $c$        $g(n)$

Closes t  
function  
 $f(n) = O(n)$

✓  $f(n) = O(n^2)$

$$2n + 3 \leq 2n + 3n$$

✓  $f(n) = O(n^3)$

$$2n + 3 \leq 5n$$

✓  $f(n) = O(2^n)$

(or)

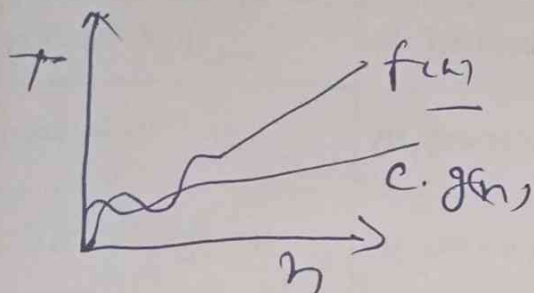
$$2n + 3 \leq 5n^2 \quad n \geq 1$$

$f(n)$        $c g(n)$

✗  $f(n) = O(\log n)$

Omega  $\Omega$  (lower bound) (6.3)

The function  $f(n) = \Omega(g(n))$



if  $\exists$  +ve constants  $c$  and  $n_0$

Such that  $f(n) \geq c * g(n) \forall n \geq n_0$

e.g.  $f(n) = 2n + 3$

$$2n + 3 \geq 1 * n \quad \forall n \geq 1$$

$$\begin{matrix} f(n) & \uparrow & c & \uparrow & g(n) \end{matrix} \quad \boxed{\therefore f(n) = \Omega(g(n))}$$

closest

$$2n + 3 \geq 1 * \log n \quad \forall n \geq 1$$

$$\therefore f(n) = \Omega(\log n)$$

$$\cancel{f(n) = \Omega(n^2)}$$

## Theta notation (Q)

(18)  
(6.4)

The function  $f(n) = O(g(n))$

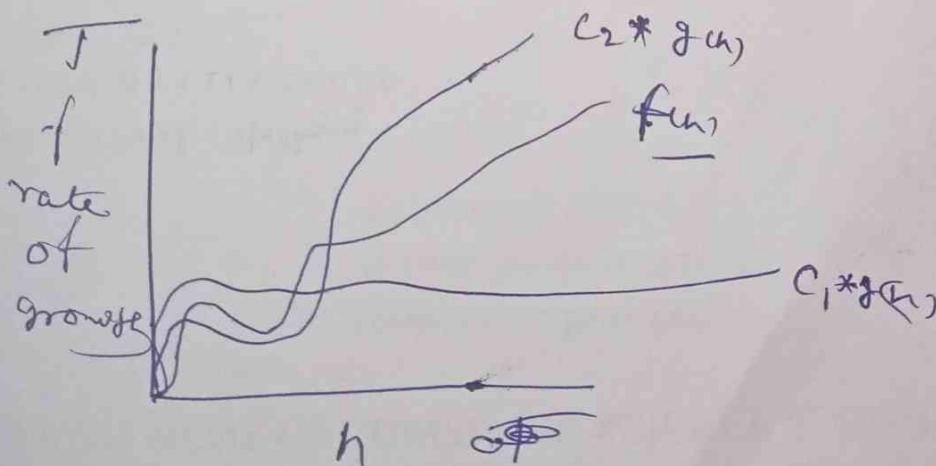
iff  $\exists$  +ve constants  
 $C_1, C_2 \neq 0$

Such that  $C_1 * g(n) \leq f(n) \leq C_2 * g(n)$

e.g.  $f(n) = 2n + 3 \therefore f(n) = O(n)$

$$\begin{array}{ccccc} 1 \times n & \leq & 2n + 3 & \leq & 5 \times n \\ C_1 g(n) & & \uparrow f(n) & & C_2 g(n) \end{array} \quad f(n)$$

Most recommended notation

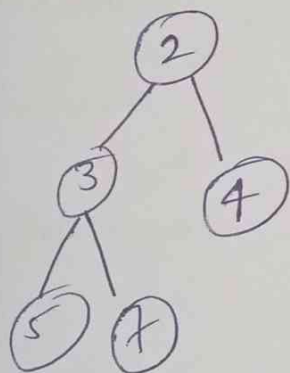




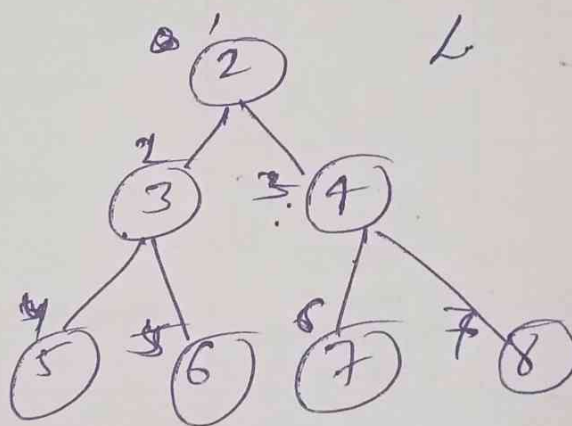
# Heap

(7)

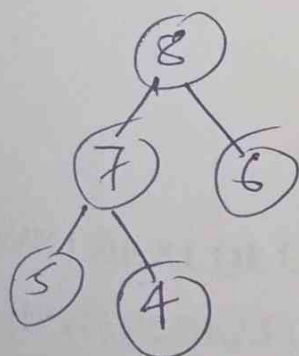
A Heap is a complete binary tree data structure that satisfies the heap property: in a min-heap the value of each child is greater than or equal to its parent and in max-heap, the value of each child is less than or equal to its parent.



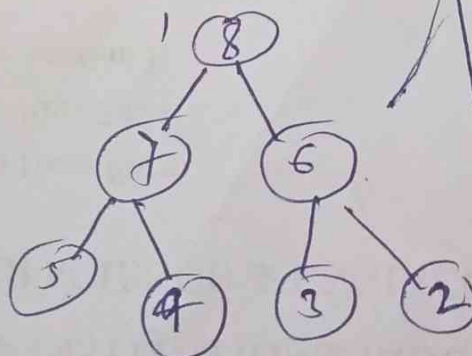
Min Heap



Min heap



Max Heap



Max Heap

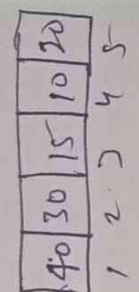
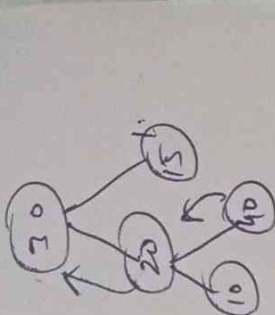
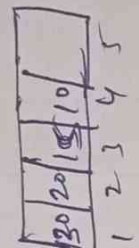
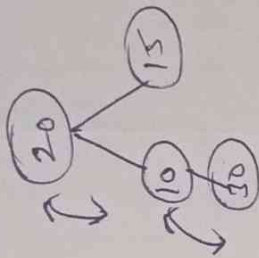
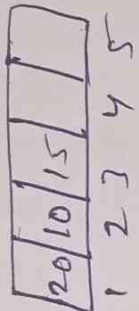
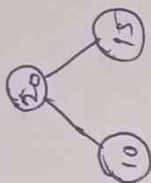
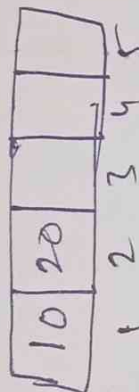
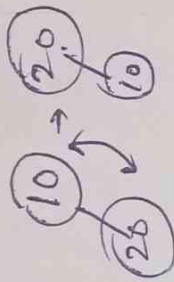
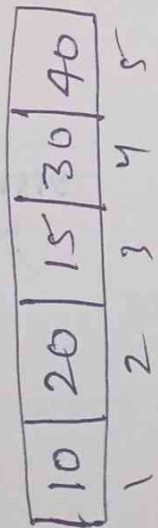
left child =  $2 \times i$   
 Right child =  $2 \times i + 1$

Parent =  $\left\lfloor \frac{i}{2} \right\rfloor$

# Heap

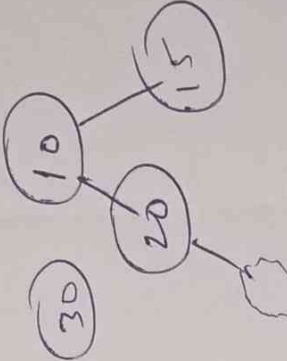
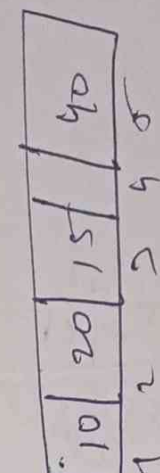
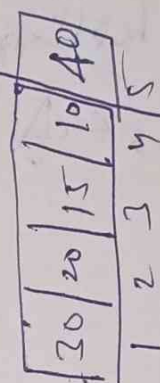
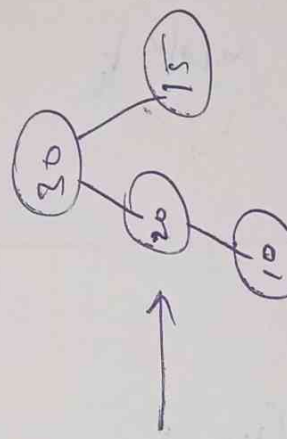
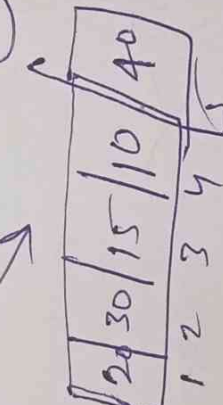
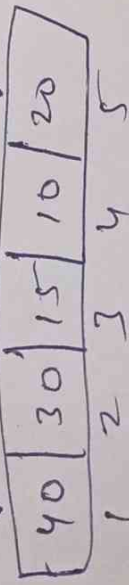
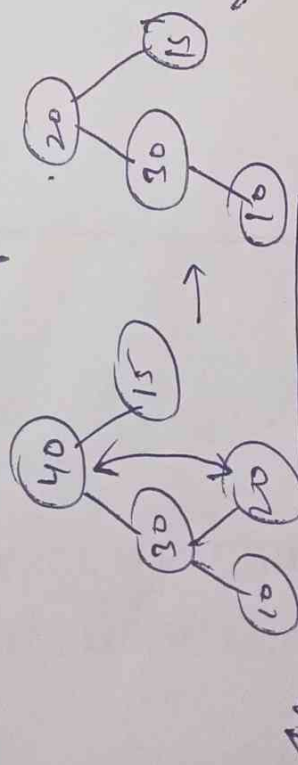
## create Heap

(10)



$n/\log n$

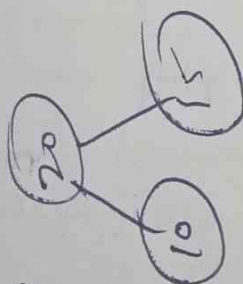
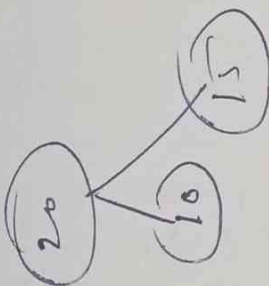
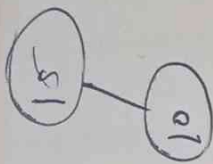
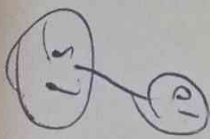
## Heap sort



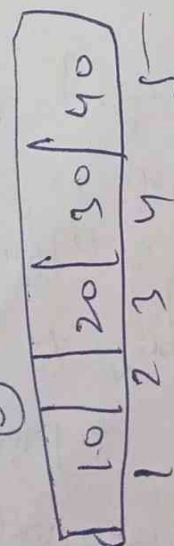
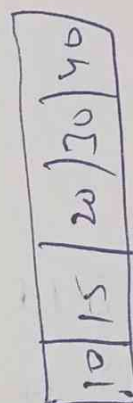
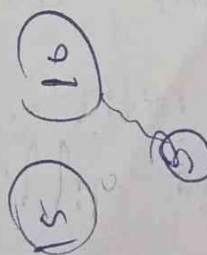
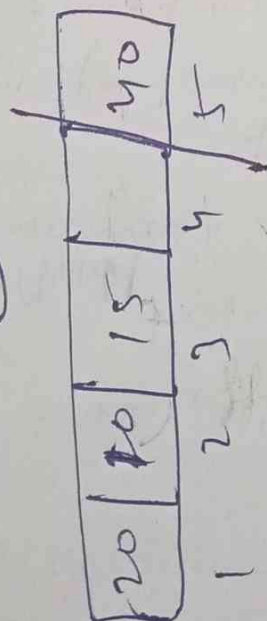
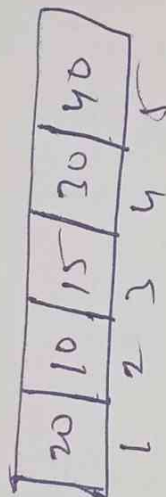
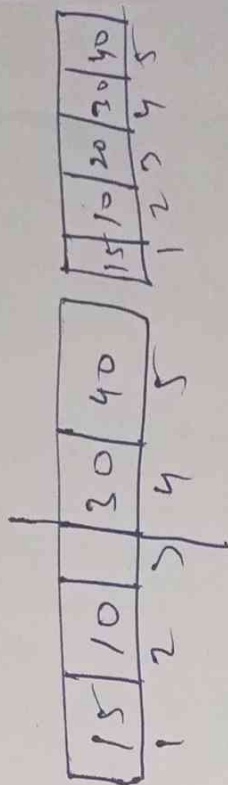
F.1

(10)



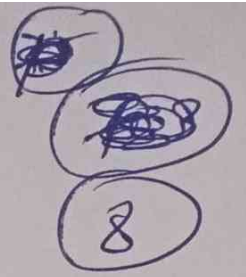


20



7.2

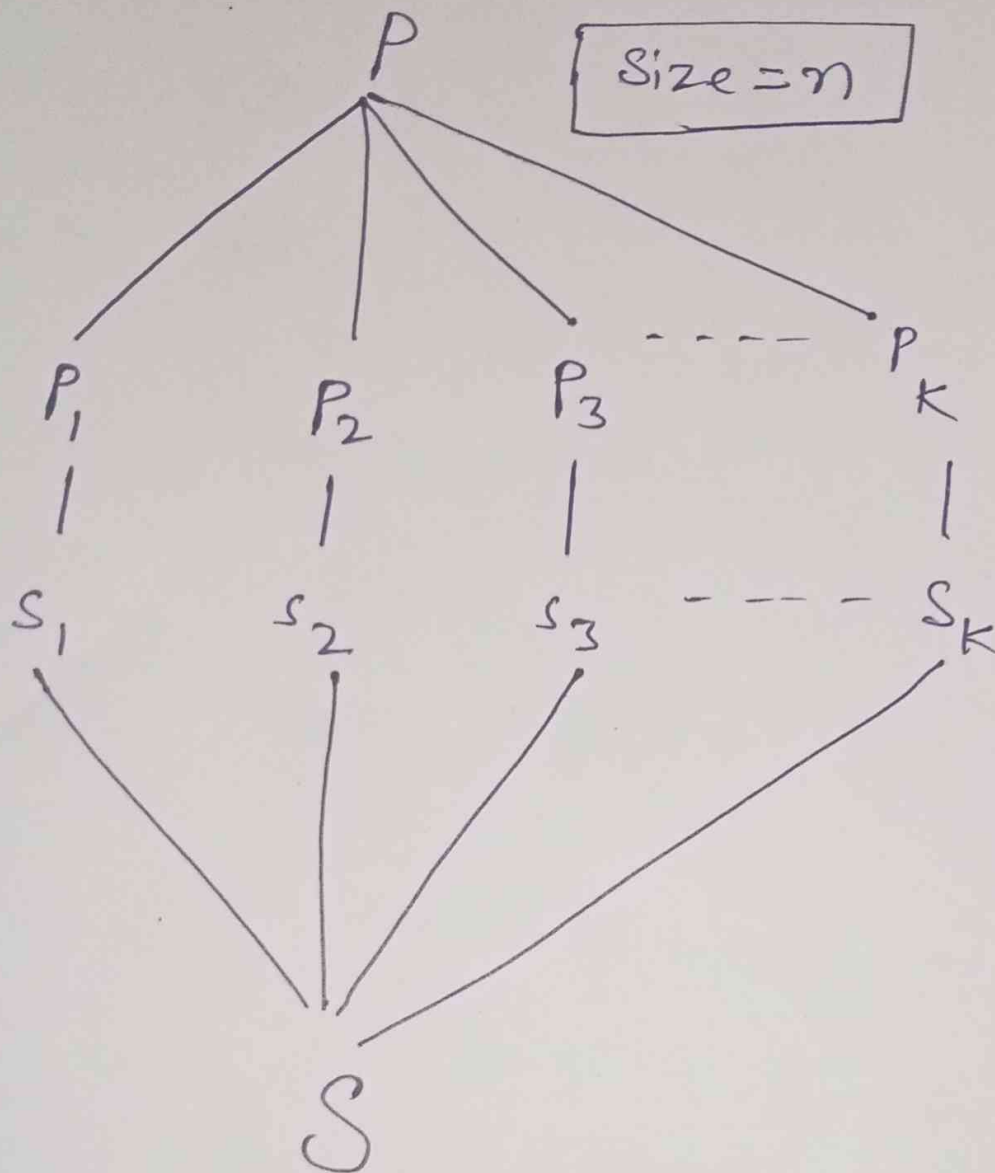
## Algorithm strategies



- ① Divide & Conquer
- ② Greedy method
- ③ Dynamic programming
- ④ Backtracking

# Divide and Conquer

9



## Divide and Conquer

~~188~~  
9.1

DAC(P)

{

if (small(P))

{

S(P);

}

else

{

divide P into  $P_1, P_2, P_3, \dots, P_k$

Apply DAC( $P_1$ ), DAC( $P_2$ ), ...

Combine (DAC( $P_1$ ), DAC( $P_2$ ), ...)

}

}

9.2

~~7.2~~

## Divide & Conquer Applied Here

- ① Binary search
- ② Merge sort
- ③ Quick sort
- ④ Strassen's matrix multiplication

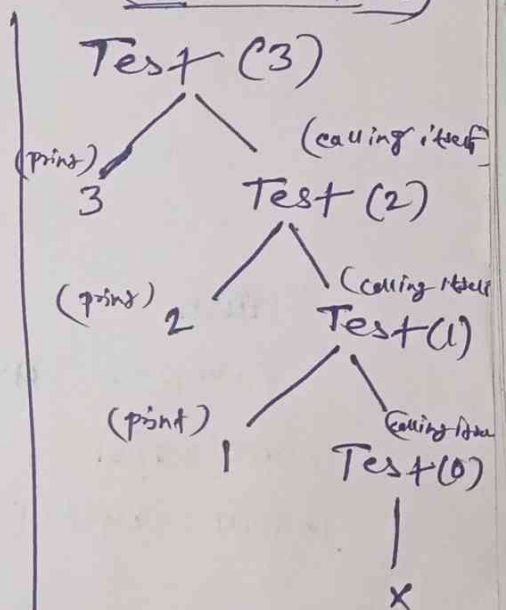
### Recurrence Relation

(Recursive Tree)

```
T(n) - void Test (int n)
{
    if (n > 0)
    {
        1 ——— printf ("%d", n);
        T(n-1) ——— Test (n-1)
    }
}
```

$$T(n) = \text{Test}(3) \\ T(n-1) + 1$$

$$T(n) = T(n-1) + 1$$



3+1

$$f(n) = n+1 \text{ (calls)} \\ n \text{ (print)}$$

O(n)



(10)

## ~~ADA~~ ~~Lab~~ Binary search Algorithm

### 1. Binary search Algorithm

- \* It works on divide & conquer strategy
- \* In divide & conquer a problem divides into subproblem & so on.
- \* ~~To perform~~ list must be sorted

(10.1)

A =

|   |   |   |    |    |    |    |     |    |    |    |    |    |    |    |
|---|---|---|----|----|----|----|-----|----|----|----|----|----|----|----|
| 3 | 6 | 8 | 12 | 14 | 17 | 25 | 29  | 31 | 36 | 42 | 47 | 53 | 55 | 62 |
| 1 | 2 | 3 | 4  | 5  | 6  | 7  | 8   | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
| ↑ |   |   |    |    |    |    | ↑   |    |    |    |    |    | ↑  |    |
| l |   |   |    |    |    |    | mid |    |    |    |    |    | h  |    |

Key = 42

$l \quad h \quad mid = \left\lfloor \frac{l+h}{2} \right\rfloor$

① → 1 15  $\frac{1+15}{2} = 8$

|    |    |    |     |    |    |    |
|----|----|----|-----|----|----|----|
| 31 | 26 | 42 | 47  | 53 | 55 | 62 |
| 9  | 10 | 11 | 12  | 13 | 14 | 15 |
| ↑  |    |    | ↑   |    |    | ↑  |
| l  |    |    | mid |    |    | h  |

② → 9 15  $\frac{9+15}{2} = 12$

|    |     |    |
|----|-----|----|
| 31 | 36  | 42 |
| 9  | 10  | 11 |
| ↑  | ↑   | ↑  |
| l  | mid | h  |

③ → 9 11  $\frac{9+11}{2} = 10$

42

④ → 10 11  $\frac{11+11}{2} = 11$

10.2

## Algorithm (Binary Search)

Bin Search ( $A, n, \text{Key}$ )

{

$l = 1;$

$h = n;$

  while ( $l \leq h$ )

  {

$\text{mid} = (l + h) / 2;$

    if ( $\text{Key} == A[\text{mid}]$ )

      return mid;

    if ( $\text{Key} < A[\text{mid}]$ )

$h = \text{mid} - 1;$

    else

$l = \text{mid} + 1;$

  }

  return 0;

}

# Binary search

10.3

10.3

$n = 15$

|       |     |    |    |    |   |   |   |    |    |    |    |    |    |    |    |     |
|-------|-----|----|----|----|---|---|---|----|----|----|----|----|----|----|----|-----|
|       | $l$ |    |    |    |   |   |   |    |    |    |    |    |    |    |    |     |
| $A =$ | -12 | -8 | -6 | -1 | 3 | 6 | 8 | 11 | 14 | 17 | 20 | 22 | 25 | 29 | 31 | $h$ |
|       | 0   | 1  | 2  | 3  | 4 | 5 | 6 | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 |     |

Key = 6

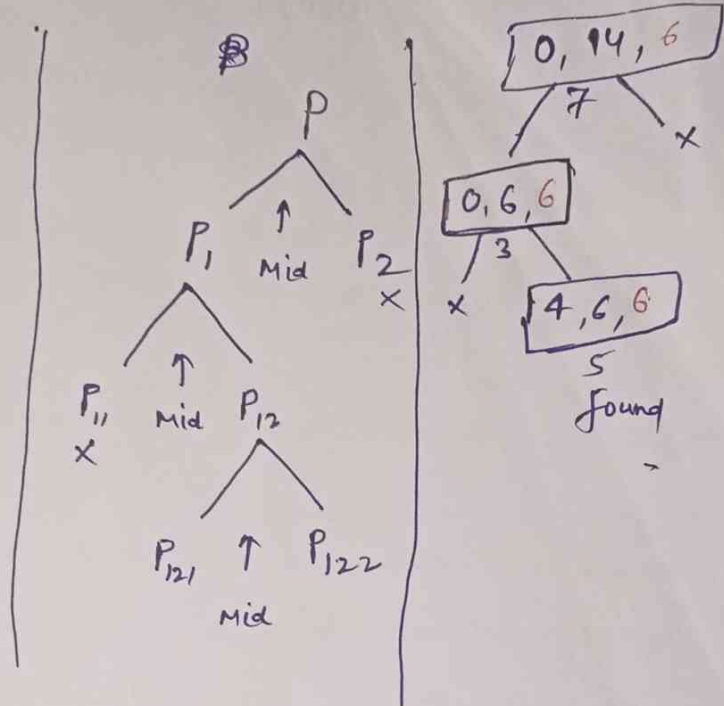
$$l \quad h \quad \text{mid} = \frac{(l+h)}{2}$$

$$0 \quad 14 \quad \frac{0+14}{2} = 7$$

$$0 \quad 6 \quad \frac{0+6}{2} = 3$$

$$4 \quad 6 \quad \frac{4+6}{2} = 5$$

found  
stop



$l = h$   
 $l < h$   
 $l > h$

Bin Srch (int l, int h, int key)

{  
if ( $l > h$ )  
return -1; // Element not found

int mid =  $(l+h)/2$ ;

if ( $key == a[mid]$ ) -

{  
return mid; // Element found

else if ( $key < a[mid]$ )

{  
return Binsrch (l, mid-1, key);

else

{  
return Binsrch (mid+1, h, key);

{  
}

(10.4)

The time Complexity of a binary Search is  $O(\log_2 n)$ .

This means that as the number of elements ( $n$ ) in a sorted array grows, the time it takes to find a target value increases very slowly.

Why is it logarithmic?

Binary search uses a divide-and-conquer strategy. Each step eliminates half of the remaining search space.

- ① Iteration 1:  $n$  elements remaining
- ② Iteration 2:  $n/2$  elements remaining
- ③ Iteration 3:  $n/4$  (or  $n/2^2$ ) elements remaining
- ④ Iteration  $k$ :  $n/2^k$  elements remaining

The search ends when the search space is reduced to 1 element ( $n/2^k = 1$ )

$$n = 2^k \Rightarrow \log_2 n \quad n = \log_2 n$$

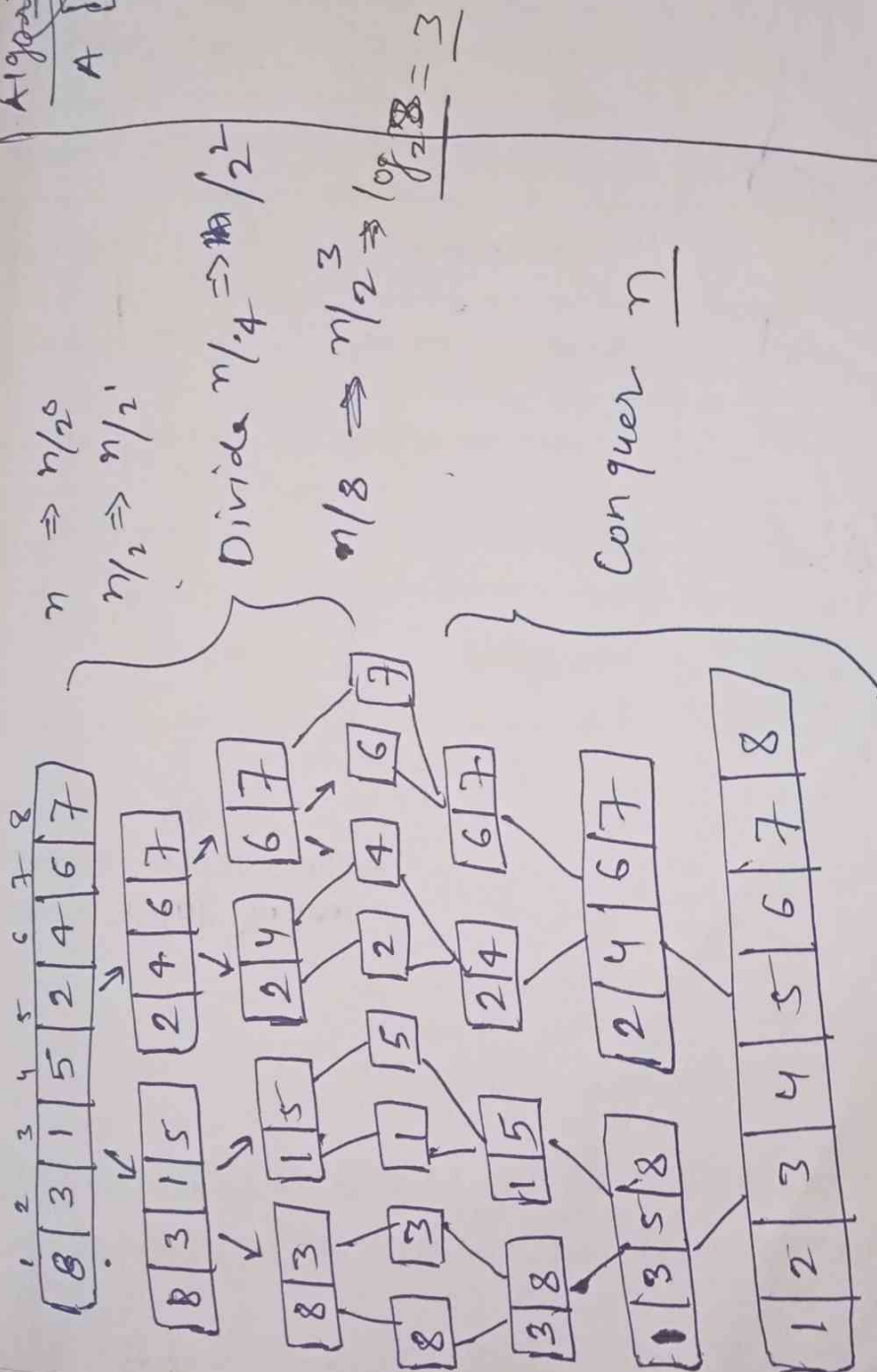
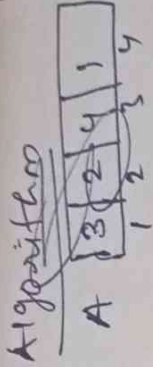


# Merge sort

11

\* Merge sort is efficient, comparison based sorting algorithm

\* Based on divide & conquer.



## Algorithm Merge\_Sort

void Merge\_Sort(int arr[], int low, int high)

1) if (low < high)

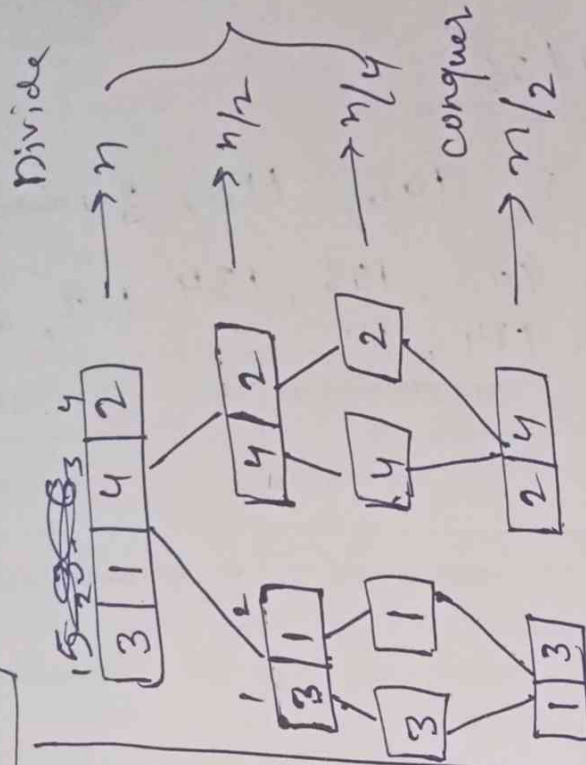
{

2) mid =  $(low + high) / 2$ ;

3) Merge\_Sort(arr, low, mid);

4) Merge\_Sort(arr, mid+1, high);

5) Merge(arr, low, mid, high);



11.1

Merge(arr, low, mid, high)

$n1\_start = mid - low + 1;$

$n2\_start = high - mid;$

Left[s~~n1~~], Right[n2];

for ( $i = 0; i < n1; i++$ )

Left[i] = arr[low + i];

for ( $j = 0; j < n2; j++$ )

Right[j] = arr[mid + i + j];

Left[n1 + 1] = 9999;

Right[n2 + 1] = 9999;

$i = 0;$

$j = 0;$

$k = low;$

while ( $i < n1 \ \& \ j < n2$ )

{

if (Left[i] <= Right[j])

{ arr[k] = Left[i];

i++;

}

else

{ arr[k] = Right[j];

j++;

k++;

}

```
// Copying from Left[]
```

```
while (i < n1)
```

```
{ arr[k] = Left[i];
```

```
  i++;
```

```
  k++;
```

```
}
```

```
// Copying from Right[]
```

```
while (j < n2)
```

```
{
```

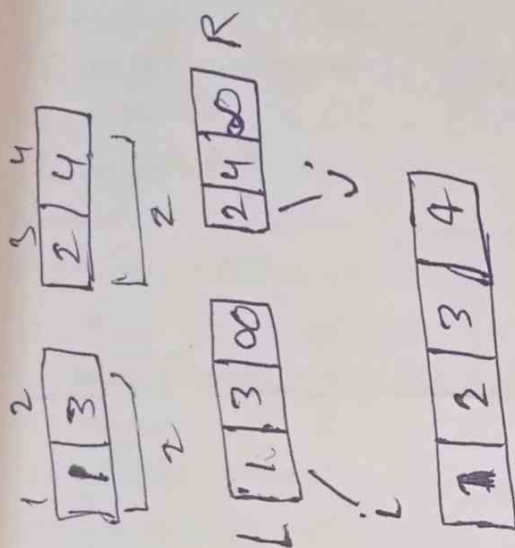
```
  arr[k] = Right[j];
```

```
  j++;
```

```
  k++;
```

```
}
```

```
}
```



Merge sort Time complexity

$$f(n) = n \log n \times n \Rightarrow n \log n$$

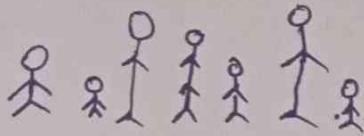
Height of Tree  
Total element to be sort

11.3



# Quick sort Algorithm

12



(10) 80 90 60 30 20

6 3 5 4 2 1 (9)

4 6 7 (10) 16 12 13 14

|   |      |    |   |    |    |   |   |   |   |          |
|---|------|----|---|----|----|---|---|---|---|----------|
|   | $l$  |    |   |    |    |   |   |   |   | $h$      |
|   | 0    | 1  | 2 | 3  | 4  | 5 | 6 | 7 | 8 | 9        |
| A | (10) | 16 | 8 | 12 | 15 | 6 | 3 | 9 | 5 | $\infty$ |
|   | $i$  |    |   |    |    |   |   |   |   | $j$      |

Pivot = 10

$i$  for  $>$   
 $j$  for  $<$

|  |      |   |   |    |    |   |   |   |    |          |
|--|------|---|---|----|----|---|---|---|----|----------|
|  | 0    | 1 | 2 | 3  | 4  | 5 | 6 | 7 | 8  | 9        |
|  | (10) | 5 | 8 | 12 | 15 | 6 | 3 | 9 | 16 | $\infty$ |
|  | $i$  |   |   |    |    |   |   |   |    | $j$      |

|      |   |   |     |    |   |   |     |    |          |
|------|---|---|-----|----|---|---|-----|----|----------|
| (10) | 5 | 8 | 9   | 15 | 6 | 3 | 12  | 16 | $\infty$ |
|      |   |   | $i$ |    |   |   | $j$ |    |          |

|      |   |   |   |     |     |    |    |    |          |
|------|---|---|---|-----|-----|----|----|----|----------|
| (10) | 5 | 8 | 9 | 3   | 6   | 15 | 12 | 16 | $\infty$ |
|      |   |   |   | $i$ | $j$ |    |    |    |          |

|      |   |   |   |   |     |     |    |    |          |
|------|---|---|---|---|-----|-----|----|----|----------|
| (10) | 5 | 8 | 9 | 3 | 6   | 15  | 12 | 16 | $\infty$ |
|      |   |   |   |   | $j$ | $i$ |    |    |          |

|   |   |   |   |   |      |     |    |    |          |
|---|---|---|---|---|------|-----|----|----|----------|
| 6 | 5 | 8 | 9 | 3 | (10) | 15  | 12 | 16 | $\infty$ |
|   |   |   |   |   | $j$  | $i$ |    |    |          |



Partition( $l, h$ )

{

    pivot =  $A[l]$ ;

$i = l, j = h$ ;

    while ( $i < j$ ) {

        do

        {

$i++$ ;

        }

        while ( $A[i] \leq \text{pivot}$ );

    do

    {

$j--$ ;

    }

    while ( $A[j] > \text{pivot}$ );

    if ( $i < j$ )

    {

        swap( $A[i], A[j]$ );

    }

}

    swap( $A[l], A[j]$ );

    return  $j$ ;

}

12.1

```

QuickSort(l, h)
{
  if (l < h)
  {
    j = Partition(l, h);
    QuickSort(l, j);
    QuickSort(j+1, h);
  }
}

```

### Another Example (Quick Sort)

Pivot

(50) 70 60 90 40 80 10 20 30 ∞  
 l j

(50) 70 60 90 40 80 10 20 30 ∞  
 l j

(50) 30 60 90 40 80 10 20 70 ∞  
 i j

(50) 30 20 90 40 80 10 60 70 ∞  
 l j

(50) 30 20 10 40 80 90 60 70 ∞  
 l j

(40 30 20 10) (50) (80 90 60 70 ∞)  
 j

Partitioning Position

Another Example (Quick sort)  
(03 elements)

12.3

(20) 10 30  $\infty$   
i j

(20) 10 30  $\infty$   
L j i

10 (20) 30  $\infty$

Another Example (Quick sort)  
02 elements

(10) 20  $\infty$   
i j

(A[i] > Pivot) (10) 20  $\infty$  (A[j] ≤ Pivot)  
j i

(10) (20)  $\infty$

Worst case

Another Example (Quick sort)



List is already sorted (Ascending order) (Worst case)

l  
 (10) 20 30 40 50 h  
 i j

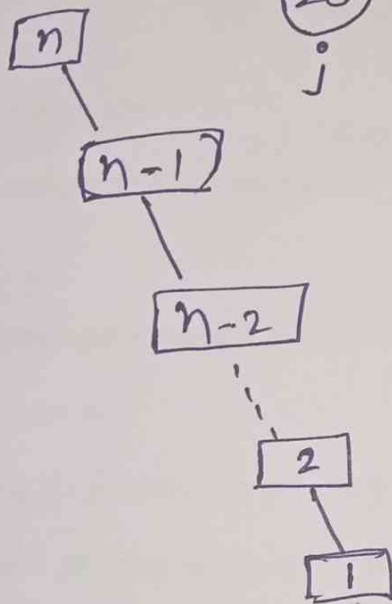
(10) 20 30 40 50 ∞ n  
 j i

(20) 30 40 50 ∞ n-1  
 j i

(30) 40 50 ∞ n-2  
 j i

(40) 50 ∞ 2  
 j i

50 ∞ 1



$$= 1 + 2 + 3 + \dots + n$$

$$= \frac{n(n+1)}{2}$$

$$= \frac{n^2 + n}{2}$$

$$= O(n^2) \text{ worst case complexity}$$

12.4

Worst case

# Another Example (Quick sort)

List sorted in descending order

(50) 40 30 20 10 ∞  
i j

(50) 40 30 20 10 ∞  
j i

10 40 30 20 50 ∞

10

(10) 40 30 20 ∞  
i j

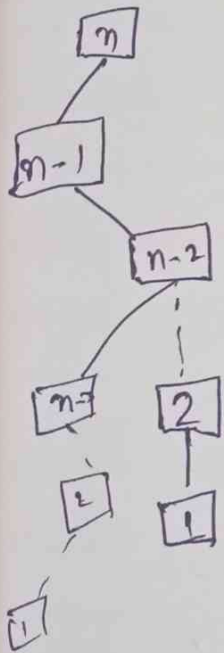
(10) 40 30 20 ∞  
j i

(40) 30 20 ∞  
i j

(40) 30 20 ∞  
j i

(20) 30 40 ∞

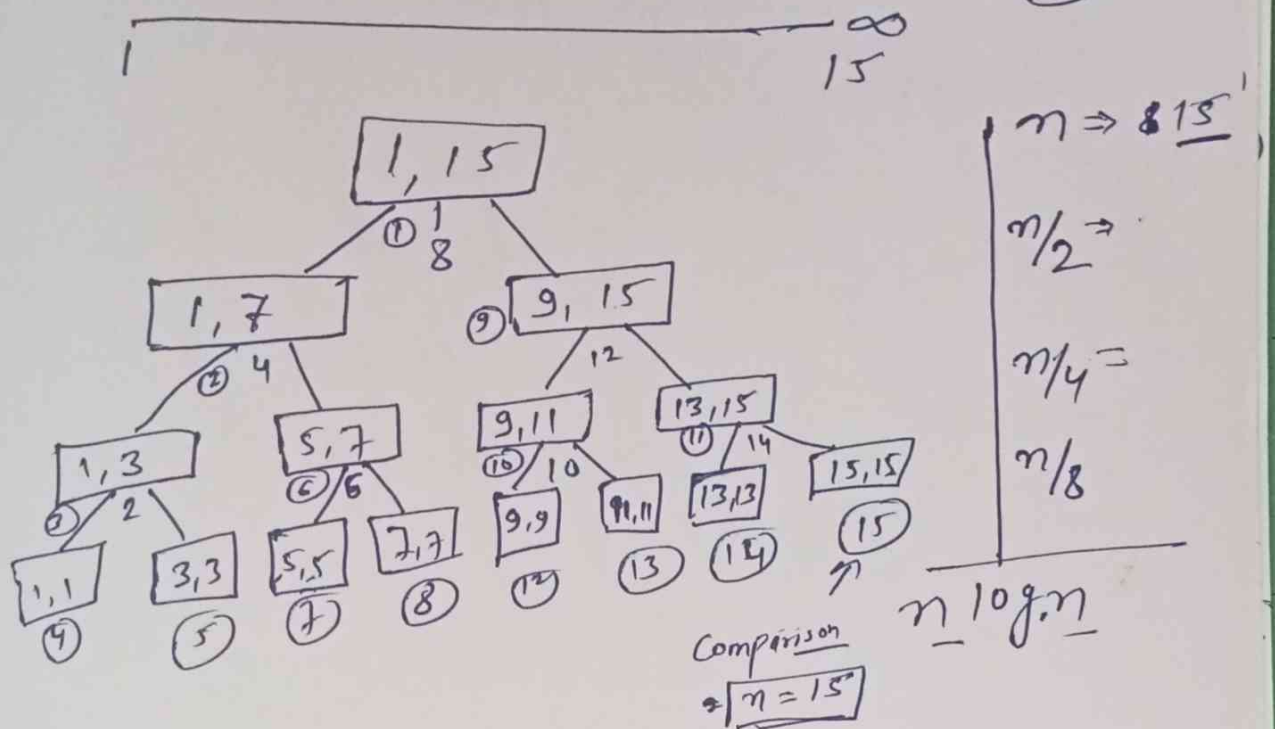
20



12.5



## Example Quick sort (Best case)



- ① Best case  $\rightarrow$  if partitioning is in middle  
Time  $\rightarrow O(n \log n)$
- ② Worst case  $\rightarrow$  if partitioning is on any end  
(already sorted)  
Time  $\rightarrow O(n^2)$
- ③ Avg. case  $\rightarrow O(n \log n)$  (you get 1-3 split or 3-1 split)

Another name of Quick sort is  
Partition exchange sort

(12.6)

# Quick sort (Average case)

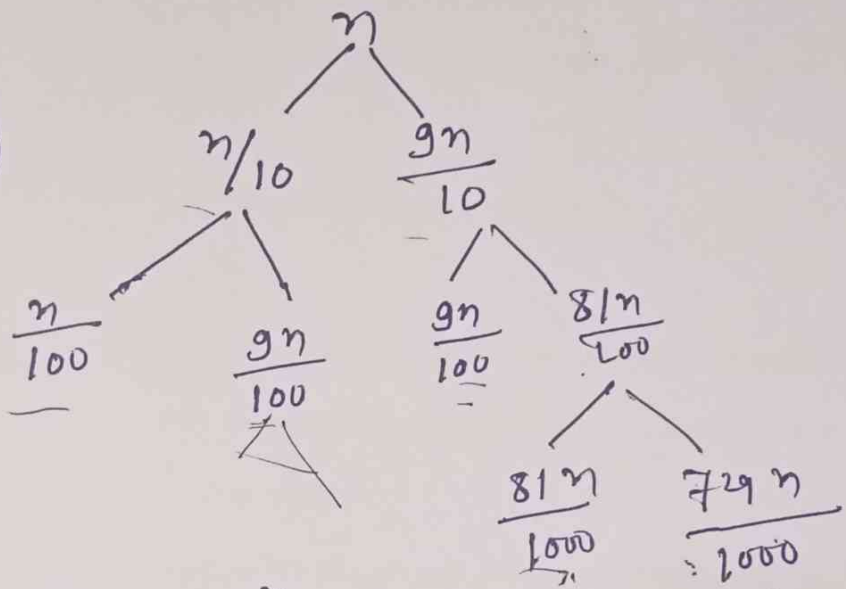
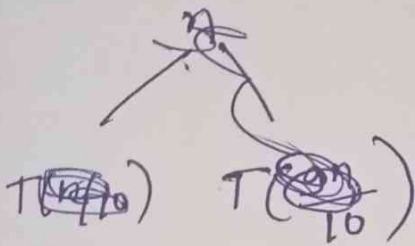
Time complexity

Best case  $n/2 \rightarrow O(n \log n)$  Time

Worst case  $n-1, 0$

Now Average case  $9:1$

$$T(n) = T(n/10) + T(9n/10) + n$$



$$n, \left(\frac{n}{10/9}\right), \left(\frac{n}{10/9}\right)^2, \left(\frac{n}{10/9}\right)^3, \dots, 1$$

$$\log_{10/9} n \Rightarrow 2^k$$

$8^2$   
 $2^3$   
 $2^3$   
2.81

$T(n) = n \log_{10/9} n$

(12.7)

Normal  
~~Strassen's~~  
~~\* not of 2x2~~

# Matrix Multiplication (DA)

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

C = m.p

2x2 2x2  
m x n m x p

2x2  
m x p

No. of column in the first matrix = No. of row in second matrix

$$C_{ij} = \sum_{k=1}^n A_{ik} * B_{kj}$$

space complexity

A -  $n^2$

B -  $n^2$

C -  $n^2$

n - 1

i = 1

j = 1

k = 1

$S(n) = 3n^2 + 4$

$O(n^2)$

n+1 — for (i=0; i ≤ n; i++)

(n+1)n — for (j=0; j ≤ n; j++)

n x n — C[i, j] = 0;

(n+1)n x n — for (k=0; k ≤ n; k++)

n x n x n — C[i, j] += A[i, k] \* B[k, j];

for =  $2n^3 + 3n^2 + 2n + 1$

$O(n^3)$

Time Complexity

$T(n) = O(n^3)$

Normal  
~~Essentials~~

13.1

Matrix Multiplication (DAC)

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}_{2 \times 2} \times B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}_{2 \times 2} \Rightarrow C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}_{2 \times 2}$$

~~$$c_{11} = a_{11} \times b_{11} + a_{21} \times b_{11}$$~~

$$c_{11} = a_{11} \times b_{11} + a_{12} \times b_{21}$$

$$c_{12} = a_{11} \times b_{12} + a_{12} \times b_{22}$$

$$c_{21} = a_{21} \times b_{11} + a_{22} \times b_{21}$$

$$c_{22} = a_{21} \times b_{12} + a_{22} \times b_{22}$$

$$A = \begin{bmatrix} a_{11} \end{bmatrix}_{1 \times 1} \quad B = \begin{bmatrix} b_{11} \end{bmatrix}_{1 \times 1}$$

$$C = \begin{bmatrix} a_{11} \times b_{11} \end{bmatrix}_{1 \times 1}$$

13.1



# Divide & Conquer

## Matrix Multiplication (DAC)

13.2

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ & A_{11} & & A_{12} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ & & & \\ a_{31} & a_{32} & a_{33} & a_{34} \\ & A_{21} & & A_{22} \\ a_{41} & a_{42} & a_{43} & a_{44} \\ & & & \end{bmatrix}$$

$$B = \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ & B_{11} & & B_{12} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ & & & \\ b_{31} & b_{32} & b_{33} & b_{34} \\ & B_{21} & & B_{22} \\ b_{41} & b_{42} & b_{43} & b_{44} \\ & & & \end{bmatrix}$$

$$\frac{4 \times 4}{2} \quad \frac{4 \times 4}{2}$$

$$\frac{4 \times 4}{2} \quad \frac{4 \times 4}{2}$$

Algorithm MM(A, B, n)

{ if (n ≤ 2)

{

C = 4 formulae;

⋮

}

else

{

mid = n/2

MM(A<sub>11</sub>, B<sub>11</sub>, n/2) + MM(A<sub>12</sub>, B<sub>21</sub>, n/2)

MM(A<sub>11</sub>, B<sub>12</sub>, n/2) + MM(A<sub>12</sub>, B<sub>22</sub>, n/2)

MM(A<sub>21</sub>, B<sub>11</sub>, n/2) + MM(A<sub>22</sub>, B<sub>21</sub>, n/2)

MM(A<sub>21</sub>, B<sub>12</sub>, n/2) + MM(A<sub>22</sub>, B<sub>22</sub>, n/2)

13.2

}

return C

$$C_{11} = A_{11} * B_{11} + A_{12} * B_{21}$$

$$C_{12} = A_{11} * B_{12} + A_{12} * B_{22}$$

$$C_{21} = A_{21} * B_{11} + A_{22} * B_{21}$$

$$C_{22} = A_{21} * B_{12} + A_{22} * B_{22}$$



~~Matrix~~ DAC  
Matrix Multiplication  
(Recurrence Relation)

$$T(n) = \begin{cases} b & n \leq 2 \\ 8T(n/2) + n^2 & n > 2 \end{cases}$$

$$8^2 = 16$$

As  $\begin{cases} a = 8 \\ b = 2 \end{cases}$   
Per master's

Theorem

$$f(n) = n^2$$

$$\boxed{n \log_b a = n \log_2 8 = n^3}$$

$$n^k = n^2$$

$$\Theta(n^{\log_b a}) \Rightarrow$$

$$\underline{\underline{\Theta(n^3)}}$$

Normal  
Matrix Multiplication

Master's Theorem

$$T(n) = aT(n/b) + f(n) \quad \begin{matrix} a \geq 1 \\ b > 1 \end{matrix}$$

It compares the cost of work done at the top level  $f(n)$  with the cost of work done at the ~~level~~ ~~for~~ leaves  $(n^{\log_b a})$

(13.3)

(13.4)

(15.2)

What Strassen has done?

$$P = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22})B_{11}$$

$$R = A_{11}(B_{12} - B_{22})$$

$$S = A_{22}(B_{21} - B_{11})$$

$$T = (A_{11} + A_{12})B_{22}$$

$$U = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$V = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = A_{11} * B_{11} + A_{12} * B_{21}$$

$$C_{12} = A_{11} * B_{12} + A_{12} * B_{22}$$

$$C_{21} = A_{21} * B_{11} + A_{22} * B_{21}$$

$$C_{22} = A_{21} * B_{12} + A_{22} * B_{22}$$

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

$$T(n) = \begin{cases} b & n \leq 2 \\ 7T(n/2) + n^2 & n > 2 \end{cases}$$

~~$$T(n) = O(n \log n)$$~~

$$\log_2 7 = 2.81$$

$$\text{for } k=2$$

$$O(n^{\log_2 7}) = O(n^{2.81})$$

(By using master's theorem  $a=7$   $b=2$ )

$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$  Strassen's Method

13.5

$P = (A_{11} + A_{22})(B_{11} + B_{22})$  ← Diagonal Multiply  
 $Q = B_{11}(A_{21} + A_{22})$  ←  $\begin{bmatrix} 11 & 12 \\ 21 & 22 \end{bmatrix} \xrightarrow{(-)} T(+)$   
 $R = A_{11}(B_{12} - B_{22})$  ←  $\begin{bmatrix} 11 & 12 \\ 21 & 22 \end{bmatrix} \xrightarrow{(-)} R$   
 $S = A_{22}(B_{21} - B_{11})$  ←  $\begin{bmatrix} 11 & 12 \\ 21 & 22 \end{bmatrix} \xrightarrow{(-)} Q(+)$   
 $T = B_{22}(A_{11} + A_{12})$  ←  $B_{11} A_{11} A_{22} B_{22}$   
 $U = (A_{21} - A_{11})(B_{11} + B_{12})$  ← Replace  $A \rightarrow B$   
 $V = (B_{21} + B_{22})(A_{12} - A_{22})$  ←  $B \rightarrow A$  Replace

Now calculate  $C_{ij}$

③  $C_{11} = P + S - T + V$  ←  $V$  first  
 ①  $C_{12} = R + T$  ← RAT formula  
 ②  $C_{21} = Q + S$  ←  $(R-1) + (T-1)$   
 ④  $C_{22} = P + R - Q + U$  ←  $U$  at last

Just write  
P @ start

## How does the Master's theorem work?

The master method is mainly derived from the [recurrence tree method](#).

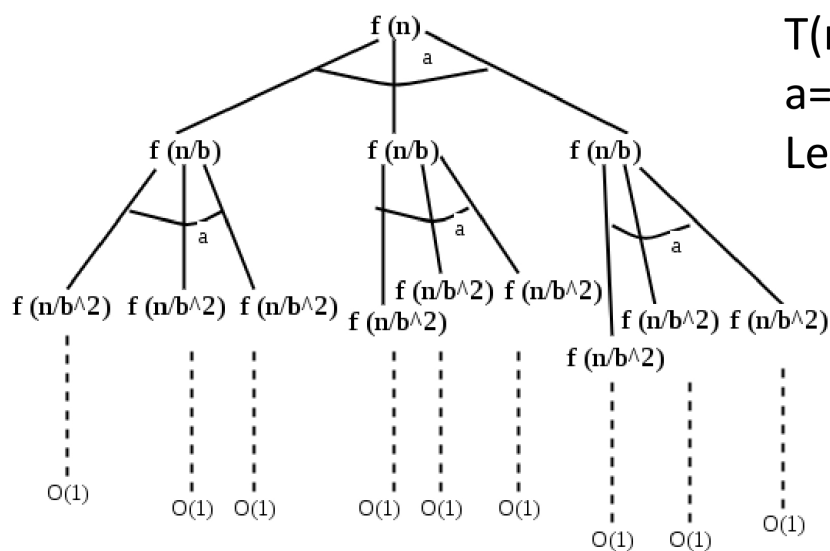
If we draw the recurrence tree of  $T(n) = aT(n/b) + f(n)$ , we can see that the work done at the root is  $f(n)$ , and work done at all leaves is  $O(n^c)$  where  $c$  is  $\log_b a$  and the height of the recurrence tree is  $\log_b n$

## Key Points

- The Master Theorem is specifically designed for divide-and-conquer recurrences
- It applies only to recurrences of the form:  $T(n) = aT(n/b) + f(n)$
- It provides a quick and reliable way to compute asymptotic time complexity
- Not all recurrences can be solved using this method
- For unsupported cases, we either use Substitution Method or Recurrence Tree Method



## Recurrence Tree Method



$$T(n) = aT(n/b) + f(n)$$

$$T(n) = 3T(n/3) + n$$

$$a=3, b=3$$

Let's assume  $n=27$

## Substitution Method

Let's Solve the following recurrence relation using substitution method  
 **$T(n)=3T(n/3)+n$**

## 1. Expand the recurrence relation

Begin by repeatedly substituting the recurrence into itself to observe the developing pattern.

- First expansion:

$$T(n) = 3T\left(\frac{n}{3}\right) + n$$

- Second expansion:

Substitute  $T(n/3) = 3T(n/9) + n/3$ :

$$T(n) = 3 \left[ 3T\left(\frac{n}{9}\right) + \frac{n}{3} \right] + n = 9T\left(\frac{n}{9}\right) + n + n = 9T\left(\frac{n}{9}\right) + 2n$$

- Third expansion:

Substitute  $T(n/9) = 3T(n/27) + n/9$ :

$$T(n) = 9 \left[ 3T\left(\frac{n}{27}\right) + \frac{n}{9} \right] + 2n = 27T\left(\frac{n}{27}\right) + n + 2n = 27T\left(\frac{n}{27}\right) + 3n$$

---

## 2. Identify the general pattern

After  $k$  substitutions, the relation takes the general form:

$$T(n) = 3^k T\left(\frac{n}{3^k}\right) + k \cdot n$$

## 3. Apply base case boundary

The iteration stops when the subproblem size reduces to the base case, typically  $T(1)$ .

Set the argument of  $T$  to 1:

$$\frac{n}{3^k} = 1 \implies n = 3^k \implies k = \log_3 n$$

## 4. Substitute $k$ back into the formula

Replace  $k$  with  $\log_3 n$  and  $3^k$  with  $n$  in the general form:

$$T(n) = n \cdot T(1) + (\log_3 n) \cdot n$$

$$T(n) = n \cdot T(1) + n \log_3 n$$

Since  $T(1)$  is a constant, the term  $n \log_3 n$  dominates the growth.

The complexity of the recurrence relation is  **$T(n) = \Theta(n \log n)$** .

### Applying the math from the image:

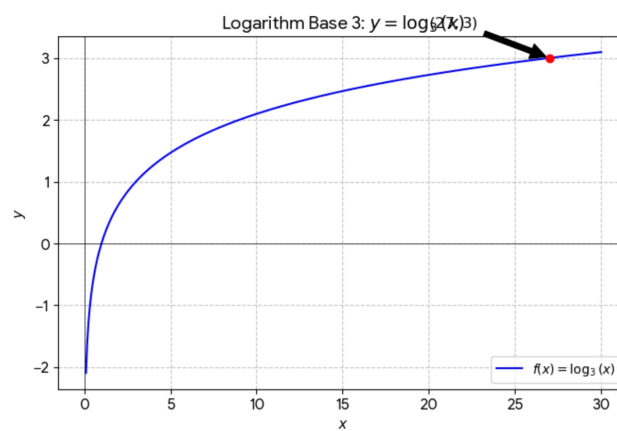
If we plug  $n = 27$  into the equations from your image, they all match up perfectly:

1. The Fraction:  $\frac{27}{3^k} = 1$

2. The Power:  $27 = 3^k$  (This asks: "3 to what power is 27?")

3. The Logarithm:  $k = \log_3 27$

Since  $3 \times 3 \times 3 = 27$ , we know that  $3^3 = 27$ . Therefore,  $k = 3$ .





Thank You